

A Proposal to add stack trace library

Significant changes to [P0881R1](#) are marked with blue.

☐ Show deleted lines from P0881R1.

I. Motivation

In the current working draft [\[N4741\]](#) there is no way to get, store and decode the current call sequence. Such call sequences are useful for debugging and post mortem debugging. They are popular in other programming languages (like Java, C#, Python).

Pretty often assertions can't describe the whole picture of a bug and do not provide enough information to locate the problem. For example, you can see the following message on out-of-range access:

```
boost/array.hpp:123: T& boost::array<T, N>::operator[](boost::array<T, N>::size_type): Assertion '(i < N)&&("out of range")' failed.  
Aborted (core dumped)
```

That's not enough information in the assert message to locate the problem without debugger.

This paper proposes classes that could simplify debugging and may change the assertion message into the following:

```
Expression 'i < N' is false in function 'T& boost::array<T, N>::operator[](boost::array<T, N>::size_type)': out of range.  
Backtrace:  
0# boost::assertion_failed_msg(char const*, char const*, char const*, char const*, long) at ../example/assert_handler.cpp:39  
1# boost::array<int, 5ul>::operator[](unsigned long) at ../../boost/array.hpp:124  
2# bar(int) at ../example/assert_handler.cpp:17  
3# foo(int) at ../example/assert_handler.cpp:25  
4# bar(int) at ../example/assert_handler.cpp:17  
5# foo(int) at ../example/assert_handler.cpp:25  
6# main at ../example/assert_handler.cpp:54  
7# 0x00007F991FD69F45 in /lib/x86_64-linux-gnu/libc.so.6  
8# 0x0000000000401139
```

II. Design Decisions

The design is based on the Boost.Stacktrace library, a popular library that does not depend on any non-standard library components.

Note about signal safety: this proposal does not attempt to provide a signal-safe solution for capturing and decoding stacktraces. Such functionality currently is not implementable on some of the popular platforms. However, the paper attempts to provide extensible solution, that may be made signal safe some day by providing a signal safe allocator and changing the stacktrace implementation details.

Note on function_info name: P0881R1 had a class named `stack_frame`. LWG found out that `stack_frame` is a bad name because `stack_frame` is usable not only in stacktraces but it also could be constructed from function pointer (not some pointer to frame) to provide information about the function.

Type name `function_info` reminds of the `type_info` which is not a very popular class. Unlike `function_info` the `type_info` class is not copyable, not assignable, not constructible by users, not hashable and not less comparable. Nevertheless we find that functionality of those types is quite common and think that `function_info` is the suitable name. Additional care was taken to not shut the door closed for making the interfaces more common in the future. Here's a list of changes that **may** be done later for making those types closer:

- add `string type_info::pretty_name()`
- add `strong_ordering type_info::operator<=>(const type_info& other) const noexcept = default;`
- add `string type_info::source_file()` and `size_t type_info::source_line()`
- add `const cahr* function_info::name()`
- enable hash specialization for `type_info`

Note on performance: during Boost.Stacktrace development phase many users requested a fast way to store stack trace, without decoding the function names. This functionality is preserved in the paper. All the `function_info` functions and constructors are lazy and won't decode the pointer information if there was no explicit request from class user.

Note on allocations: initial implementations of Boost.Stacktrace were not using allocator and all the frames were placed inside a fixed size internal storage. That was a mistake! Sometimes the most important information is located at the bottom of the stack. For example if you run Boost.Test, then the test name will be located low on the stack. With a fixed size storage the bottom of the stack could be lost along with the information.

Current design assumes that by default users wish to see the whole stack and OK with dynamic allocations, because do not construct stacktrace in performance critical places. For those users, who wish to use stacktrace on a hot path or in embedded environments `basic_stacktrace` allows to provide a custom allocator that allocates on the stack or in some other place, where users thinks it is appropriate.

Note on returning `std::string` and not having `noexcept` on `function_info::source_line()`: Unfortunately this is a necessity on some platforms, where getting source line requires allocating or where source file name [returned into](#) a storage [provided by user](#).

Note on expected implementation: We assume that Standard Library implementations would allow to disable/enable gathering stack traces by a compiler switch that does not require recompiling the whole project. In other words, we expect to see a compiler option like `-fno-stacktrace` or `libstacktrace/lib_stacktrace_noop` libraries with the same ABI that would force the constructor of the `basic_stacktrace` to do nothing. This feature is implemented in Boost.Stacktrace and is highly requested in big projects.

III. Significant changes to review

LEWG:

- `stack_frame` was renamed to `function_info`.
- `native_ptr_t` is now an object pointer rather than a function pointer. The intent is to require some kind of `const void*` that is **not** callable but could store function pointers.
- Added hash support.
- LWG requested `rbegin` like functions for `basic_stacktrace`, so now `basic_stacktrace` satisfies the requirements of an allocator-aware container, of a sequence container and reversible container except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions and default constructor are different from those required for a container.
- Added `template<class MemberFunc> explicit function_info(MemberFunc addr) noexcept;`. Does LEWG wish to see this function?
- How about moving `function_info` into a separate header `<function_info>?`

Points of special interest for CWG:

- `native_ptr_t` description
- `stacktrace` definition in `[stacktrace]` and member functions in `[function_info.query]`. Wording should not prevent any optimizations.

IV. Wording Intent

Key features that should be preserved by implementations:

- All the functions are lazy and do not query the pointer information if there was no explicit request.
- No fixed max size for trace - all the available invokers must be stored in a `stacktrace`.
- Implementations should allow to disable/enable gathering stacktraces by a link-time switch.
- Stacktracing does not prevent any of the optimizations.
- No info for a pointer is OK (it's not an error, do not throw!).
- `function_info::pretty_name()` should return a demangled function signature if possible.
- `to_string(stacktrace)` should query information from debug symbols, symbol export tables and return demangled signatures if possible.

V. Wording

23.? Stacktrace

[stacktrace]

Define `INVOKER(x)` as a function that returns the function that invoked function `x`; `NTH_INVOKERn(x)` as a function `NTH_INVOKERn-1(INVOKER(x))` for `n > 0` and `NTH_INVOKER0(x)` as `INVOKER(x)`. A sequence of functions `(f0, ..., fm)` called *stacktrace*, where:

- `f0` is the current function or the result of applying `NTH_INVOKERn1(x)` to the current function with implementation-defined `n1`,
- for each `j` in range `(0, m]` function `fj` is the result of applying `NTH_INVOKERnj(x)` to the `fj-1` with implementation-defined `nj` and `m`,
- `fm` also is the `main` function or is the result of applying `NTH_INVOKERnm(x)` to the `main` function with implementation-defined `nm`; `m` could be zero if current function is the `main` function.

This subclause describes components that C++ programs may use to store the stacktrace of the current thread of execution, query information about the stored stacktrace or particular function at runtime.

23?.1 Header `<stacktrace>` synopsis

[stacktrace.syn]

```
namespace std {
    // 23?.2, class function_info
    class function_info;

    // 23?.3, class basic_stacktrace
    template<class Allocator>
    class basic_stacktrace;

    // basic_stacktrace typedef names
    using stacktrace = basic_stacktrace<allocator<function_info>>;

    // 23?.4, non-member functions
    void swap(function_info& a, function_info& b) noexcept;

    template<class Allocator>
    void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b);

    template<class Allocator>
    string to_string(const basic_stacktrace<Allocator>& st);

    string to_string(const function_info& f);

    template<class charT, class traits, class Allocator>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);
```

```

template<class charT, class traits>
basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>& os, const function_info& f);

// 23.?.5, hash support
template<class T> struct hash;
template<> struct hash<function_info>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
}

```

23.?.2 Class function_info

[function_info]

```

namespace std {
class function_info {
public:
    // member typedefs
    using native_ptr_t = implementation-defined;

    // 23.?.2.1, construct/copy/assign/destroy
    constexpr function_info() noexcept;
    constexpr explicit function_info(native_ptr_t addr) noexcept;
    template<class Func> explicit function_info(Func* addr) noexcept;
    template<class MemberFunc> explicit function_info(MemberFunc addr) noexcept;

    constexpr function_info(const function_info& other) noexcept = default;
    constexpr function_info& operator=(const function_info& other) noexcept = default;
    ~function_info() = default;

    // 23.?.2.2, observers
    constexpr native_ptr_t address() const noexcept;
    constexpr explicit operator bool() const noexcept;

    // 23.?.2.3, query
    string pretty_name() const;
    string source_file() const;
    size_t source_line() const;

    // 23.?.2.4, comparison
    constexpr strong_ordering operator<=>(const function_info& other) const noexcept = default;

    // 23.?.2.5, modifiers
    void swap(function_info& other) noexcept;

private:
    native_ptr_t ptr_; // exposition only
};
}

```

An object of class `function_info` stores an address of any function type or an address of any member function type or it has no value. `function_info` provides operations for querying information not deducible from function type.

`native_ptr_t` is an implementation-defined object pointer type that designates a function or member function. Pointer arithmetic on `native_ptr_t` is ill-formed.

23.?.2.1 function_info construct/copy/assign/destroy

[function_info.cons]

```
constexpr function_info() noexcept;
```

Ensures: `ptr_ == nullptr`.

```
constexpr explicit function_info(native_ptr_t addr) noexcept;
```

Ensures: `ptr_ == addr`.

```
template<class Func> explicit function_info(Func* addr) noexcept;
```

Constraints: `Func*` is a function pointer.

Effects: Stores address of a function pointed by `addr` in `ptr_` in an implementation-defined manner.

```
template<class MemberFunc> explicit function_info(MemberFunc addr) noexcept;
```

Constraints: `MemberFunc` is a member function pointer.

Effects: Let `T` be the `MemberFunc` class type. Stores the address of the member function pointed by `addr` for the object of type `T` in `ptr_` in an implementation-defined manner or `nullptr` if implementation limitations prevent such conversion.

[Note: For virtual functions it stores the actual function pointer from vtable of the `T` into the `ptr_`. - end note]

[Example:

```

struct base {
    void no_virt();
    virtual void virt() = 0;
}

```

```

};
struct derived: base {
    void no_virt();
    void virt() override;
};

function_info(&base::no_virt).source_line(); // returns information for base::no_virt if possible
function_info(&base::virt).source_line(); // returns information for base::virt if possible
function_info(&derived::no_virt).source_line(); // returns information for derived::no_virt if possible

using base_ptr_t = void(base::*)();
base_ptr_t base_ptr = &derived::virt;

// returns information for base::virt if possible
function_info(base_ptr).source_line();

```

- end example]

23.?.2.2 function_info observers

[function_info.obs]

```
constexpr native_ptr_t address() const noexcept;
```

Returns: ptr_.

```
constexpr explicit operator bool() const noexcept;
```

Returns: ptr_ != nullptr.

23.?.2.3 function_info query

[function_info.query]

[Note: All the function_info query functions treat errors other than memory allocation errors (like errors during filesystem operations) as "no information available" and do not throw in that case. - end note]

```
string pretty_name() const;
```

Returns: Platform-specific signature of the function pointed by ptr_ or function with functionality close to ptr_, if present and such information is available; empty string otherwise.

Throws: bad_alloc if memory for the internal data structures or the resulting string cannot be allocated.

```
string source_file() const;
```

Returns: The presumed name of the source file [cpp.predefined] with the definition of the wrapped function, if present and such information is available; empty string otherwise.

Throws: bad_alloc if memory for the internal data structures or the resulting string cannot be allocated.

```
size_t source_line() const;
```

Returns: The presumed line number (within the source file) [cpp.predefined] with the beginning of the definition of the wrapped function, if present and such information is available; 0 otherwise.

Throws: bad_alloc if memory for the internal data structures cannot be allocated.

23.?.2.4 function_info comparison

[function_info.comp]

```
constexpr strong_ordering operator<=>(const function_info& other) const noexcept = default;
```

Returns: result of implementation specific comparison of this->ptr_ and other.ptr_.

23.?.2.4 function_info modifiers

[function_info.mod]

```
void swap(function_info& other) noexcept;
```

Effects: Exchanges the contents of *this and other.

23.?.3 Class template basic_stacktrace

[stacktrace.basic.template]

```

namespace std {
    template<class Allocator>
    class basic_stacktrace {
    public:
        using value_type = function_info;
        using const_reference = const value_type&;
        using reference = value_type&;
        using const_iterator = implementation-defined;
        using iterator = const_iterator;
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;
        using difference_type = typename iterator_traits<iterator>::difference_type;
        using size_type = typename allocator_traits<Allocator>::size_type;
        using allocator_type = Allocator;
    };
}

```

```

// 23.?.3.1, construct/copy/destroy
basic_stacktrace() noexcept;
explicit basic_stacktrace(const allocator_type& alloc) noexcept;
basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;

basic_stacktrace(const basic_stacktrace& other) = default;
basic_stacktrace(basic_stacktrace&& other) noexcept = default;
basic_stacktrace& operator=(const basic_stacktrace& other) = default;
basic_stacktrace& operator=(basic_stacktrace&& other) = default;
~basic_stacktrace() = default;

// 23.?.3.2, observers
allocator_type get_allocator() const;

const_iterator begin() const noexcept;
const_iterator end() const noexcept;
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

explicit operator bool() const noexcept;
[[nodiscard]] bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

const_reference operator[](size_type ) const;
const_reference at(size_type ) const;

// 23.?.3.3, comparisons
template <class Allocator2>
strong_ordering operator<=>(const basic_stacktrace< Allocator2 >& rhs) const noexcept;

// 23.?.3.4, modifiers
void swap(basic_stacktrace& other);

private:
vector<value_type, allocator_type> frames; // exposition only
};
}

```

The `basic_stacktrace` template class stores the stacktrace of the current thread of execution on construction and provides access to the stored stacktrace.

The class template `basic_stacktrace` satisfies the requirements of an allocator-aware container, of a sequence container and reversible container (21.2.1, 21.2.3) except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions and default constructor are different from those required for a container.

23.?.3.1 `basic_stacktrace` construct/copy/destroy

[stacktrace.basic.cons]

```

basic_stacktrace() noexcept;
explicit basic_stacktrace(const allocator_type& alloc) noexcept;

```

Effects: Stores the stacktrace of the current thread of execution in frames. alloc is passed to the frames constructor.

[Note: If `!!*this` then `frames.front()` is the `function_info` for the current function or for the the result of applying $\text{NTH_INVOKER}_{n1}(x)$ to the current function with implementation-defined $n1$; `frames.back()` is the `function_info` for the main function or is the `function_info` for the result of applying $\text{NTH_INVOKER}_{nm}(x)$ to the main function with implementation-defined nm - end note]

Ensures: `!!*this` if failed to store stacktrace of the current thread of execution; `!!*this` otherwise.

```

basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;

```

Effects: Stores subrange $[f_{\text{skip}}, f_{\text{min}(\text{skip} + \text{max_depth}, n)]$ of the stacktrace of the current thread of execution in frames, where $(f_0, \dots, f_n)$ represents a whole stacktrace. alloc is passed to the frames constructor.

Ensures: `!!*this` if failed to store non-empty stacktrace of the current thread of execution; `!!*this` otherwise.

23.?.3.2 `basic_stacktrace` observers

[stacktrace.basic.obs]

```

allocator_type get_allocator() const;

```

Returns: `frames.get_allocator()`.

```

const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;

```

Returns: `frames.cbegin()`.

```
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

Returns: frames.cend().

```
const_iterator rbegin() const noexcept;
const_iterator crbegin() const noexcept;
```

Returns: frames.crbegin().

```
const_iterator rend() const noexcept;
const_iterator crend() const noexcept;
```

Returns: frames.crend().

```
explicit operator bool() const noexcept;
```

Returns: !frames.empty().

```
[[nodiscard]] bool empty() const noexcept;
```

Returns: frames.empty().

```
size_type size() const noexcept;
```

Returns: frames.size().

```
size_type max_size() const noexcept;
```

Returns: frames.max_size().

```
const_reference operator[](size_type frame_no) const;
```

Expects: frame_no < size().

Returns: frames[frame_no].

Throws: Nothing.

```
const_reference at(size_type frame_no) const;
```

Throws: out_of_range if frame_no >= size().

Returns: frames[frame_no].

23.?.3.3 basic_stacktrace comparisons

[stacktrace.basic.comp]

```
template <class Allocator2>
strong_ordering operator<=>(const basic_stacktrace< Allocator2 >& rhs) const noexcept;
```

Returns: this->size() <= rhs.size() if this->size() != rhs.size(). lexicographical_compare_3way(this->begin(), this->end(), rhs.begin(), rhs.end()) otherwise.

23.?.3.4 basic_stacktrace modifiers

[stacktrace.basic.mod]

```
void swap(basic_stacktrace& other);
```

Effects: Exchanges the contents of *this and other.

23.?.4 Non-member functions

[stacktrace.nonmembers]

```
void swap(function_info& a, function_info& b) noexcept;
```

Effects: Equivalent to a.swap(b).

```
template<class Allocator>
void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b);
```

Effects: Equivalent to a.swap(b).

```
template<class Allocator>
string to_string(const basic_stacktrace<Allocator>& st);
```

Returns: An implementation-specific multiline string with information about each function of a stacktrace.

```
string to_string(const function_info& f);
```

Returns: An implementation-specific string with information about the stored in f function address.

```
template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);
```

Effects: As if by `os << to_string(bt);`
Returns: `os`.

```
template<class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const function_info& f);
```

Effects: As if by `os << to_string(f);`
Returns: `os`.

23.?.5 Hash support

[stacktrace.hash]

```
template<> struct hash<function_info>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
```

The specialization is enabled (23.14.15).

Feature-testing macro

Add a row into the "Standard library feature-test macros" table [support.limits.general]:

__cpp_lib_stacktrace	201811	<stacktrace>
----------------------	--------	--------------

VI. Acknowledgements

Many thanks to Jens Maurer, JF Bastien and Marshall Clow for pointing out many issues in the early wordings.
Many many thanks to all the people who participated in the LWG meeting on 20th of August and reviewed early version of the wording.